

Coprocessador RSA · AMBA 3 APB · TSMC 28 nm

Design Review — RSA_APB

Autores
Matheus Grossi · Milena Resmer

Gerente
Alcides Costa

Entregáveis do projeto

Critérios de aceitação

ID	Entregável	Requisito de aceitação
E01	Coprocessador RSA de 32 bits e respectivo testbench em SystemVerilog	Código-fonte disponível e funcional
E02	Relatório de equivalência lógica entre as implementações RSA de 4 bits em Verilog e SystemVerilog	Equivalência lógica comprovada
E03	Relatório de simulação funcional do `rsa_core32b`	Testes executados com sucesso
E04	Ambiente de verificação UVM para a interface `rsa_apb`	Ambiente funcional e executável
E05	Relatório de simulação funcional da interface RSA_APB de 32 bits	Testes funcionais aprovados
E06	Relatório de cobertura funcional da interface RSA_APB de 32 bits	Métricas apresentadas e correlacionadas ao plano de verificação
E07	Gate-level netlist mapeado para a tecnologia TSMC de 28 nm	Netlist gerado corretamente pelo Cadence Genus
E08	Relatório de análise estática de temporização (STA)	Ausência de violações de setup nos caminhos analisados (TNS = WNS = 0)
E09	Relatório de simulação pós-síntese da interface RSA_APB, com anotação de atrasos SDF	Testes aprovados, sem violações de temporização reportadas durante a simulação
E10	Scripts de automação do fluxo de desenvolvimento	Execução reproduzível das etapas e geração dos respectivos artefatos

Rascunho da arquitetura

Concepção inicial do periférico RSA_APB

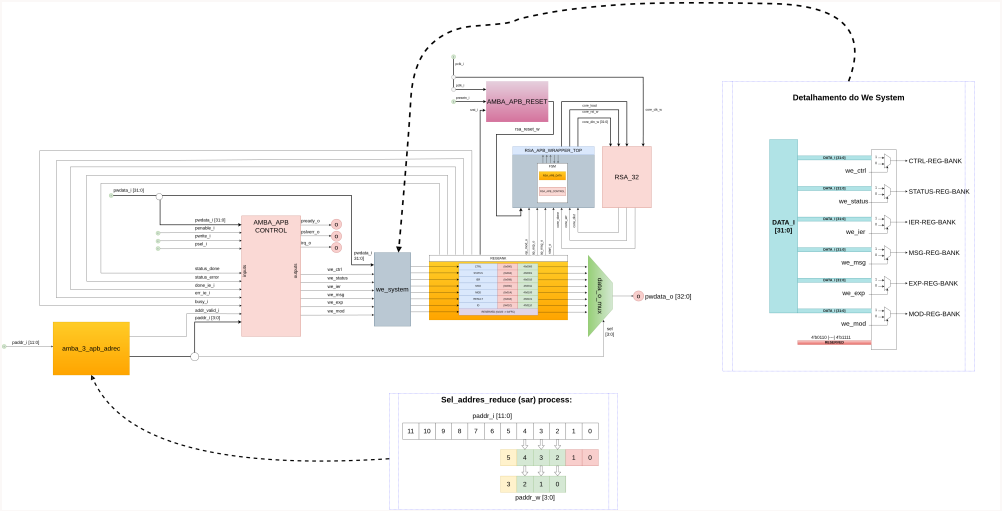
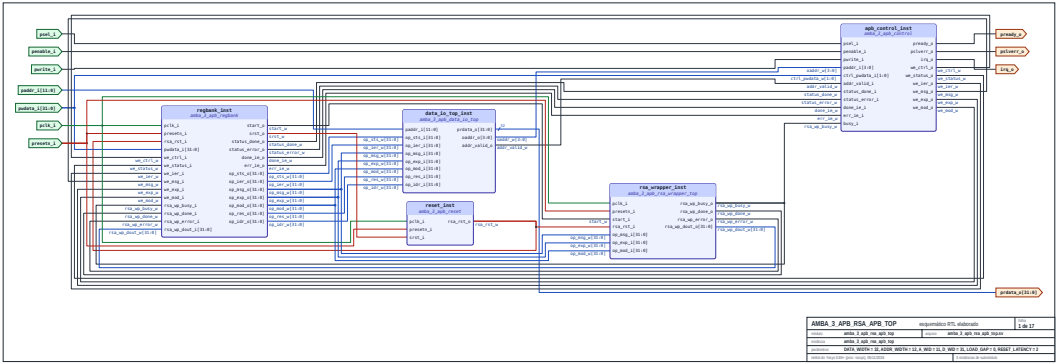


Diagrama de blocos do topo

`amba_3_apb_rsa_apb_top` — esquemático RTL elaborado



E01 — Coprocessador RSA de 32 bits e testbench

Entregável

Coprocessador RSA de 32 bits e respectivo testbench em SystemVerilog

Requisito de aceitação

Código-fonte disponível e funcional

Pergunta da revisão: *O módulo `rsa_core32b` possui um testbench funcional?*

E02 — Equivalência lógica RSA 4 bits

Cadence Conformal LEC · golden Verilog × revised SystemVerilog

Entregável

Relatório de equivalência lógica entre as implementações RSA de 4 bits em Verilog e SystemVerilog

Requisito de aceitação — ATENDIDO

Equivalência lógica comprovada

Pergunta da revisão: *O RSA de 4 bits do cliente (Verilog) é logicamente equivalente à versão em SystemVerilog? Apresente as evidências.*

EQUIVALÊNCIA COMPROVADA — PASS

94 de 94 pontos equivalentes · 0 não equivalentes · 0 abortados

Golden	`src/vlog/rsa_core.v` (`-verilog2k`)
Revised	`src/sv/rsa_core.sv` (`-systemverilog`)
Topo	`rsa_core`
Ferramenta	Conformal 23.20-s200
Modo	`lec`, `compare_effort high`
Pontos	`add_compared_points -all`
Saída	`Compare Results: PASS` (código 0)

▶ Evidências E02 no GitHub

E02 — Equivalência lógica RSA 4 bits

Mapeamento, comparação e ressalvas

Estrutura elaborada (idêntica nas duas versões)

Métrica	Golden	Revised
Módulos de projeto	5	5
Entradas / saídas primárias	7 / 7	7 / 7
Primitivas (nível de bit)	866	866
Flip-flops	88	88

Comparação formal

Pontos	PO	DFF	Total
Equivalentes	7	87	**94**
Não equivalentes	0	0	0
Abortados	0	0	0

- 88 DFFs por versão: 87 mapeados e provados; **1 inalcançável** (*unreachable*) em ambas — não é ponto não equivalente.
- `report_verification`: 0 opções não padrão, 0 verificação incompleta, 0 modificação manual, 0 ambiguidade.
- Avisos de lint idênticos nos dois lados (largura de operandos, `case` sem `default`).

Escopo

LEC prova que as duas descrições são equivalentes entre si; não prova conformidade do RSA com a especificação nem ausência de bugs presentes em ambas.

▶ **Evidências E02 no GitHub**

E03 — Simulação funcional do `rsa_core32b`

Entregável

Relatório de simulação funcional do `rsa_core32b`

Requisito de aceitação

Testes executados com sucesso

Pergunta da revisão: *Execute a simulação do `rsa_core32b` e mostre os resultados.*

E04 — Ambiente de verificação UVM

Entregável

Ambiente de verificação UVM para a interface `rsa_apb`

Requisito de aceitação

Ambiente funcional e executável

E05 — Simulação funcional do RSA_APB

RTL · Xcelium 24.03 · 100 MHz

Entregável

Relatório de simulação funcional da interface RSA_APB de 32 bits

Requisito de aceitação — ATENDIDO

Testes funcionais aprovados

Execução	`2026-11-05_15-10-39`
Topo / TB	`amba_3_apb_rsa_apb_top` / `testbench.sv`
Parâmetros	32 bits dados · 12 bits endereço · `LOAD_GAP=1`
Clock	10 ns (100 MHz) · modelo RTL, sem SDF

TEST PASS — 398 / 398

0 falhas · 0 mensagens

*E/*F · 134,98 μs simulados

Plano executado

$$N_{\text{checks}} = \underbrace{8}_{\text{iniciais}} + \underbrace{30 \times 8}_{\text{banco de reg.}} + \underbrace{10 \times 15}_{\text{operações RSA}} = 398$$

Cada operação RSA verifica carga e preservação dos operandos, BUSY/DONE, `RESULT`, IRQ e retorno a IDLE; referência: $MSG^{EXP} \bmod MOD$ calculada no testbench.

▶ Evidências E05 no GitHub

E05 — Simulação funcional do RSA_APB

Vetores RSA e cobertura de código

Operações RSA (esperado = lido)

#	MSG	MOD	EXP	RESULT
1	0x00000041	0x00000ca1	17	0x00000ae6
2	0x0000007b	0x00000ca1	7	0x00000b34
3	0x00000000	0x00000061	5	0x00000000
4	0x00000001	0x00000101	13	0x00000001
5	0x0000002a	0x00000061	0	0x00000001
6	0x12345678	0x0000fff1	1	0x00006793
7	0x80000001	0xfffffffb	2	0x4000000b
8	0xffffffff	0xfffffffb	3	0x00000040
9	0x075bcd15	0x00010001	7	0x00008a8e
10	0xfedcba98	0x8000000b	17	0x14adadfc

Inclui casos de canto: MSG$= 0$, MSG$= 1$, EXP$= 0$, EXP$= 1$, MSG e MOD próximos de 2^{32}.

Cobertura IMC (DUT, *Covered*)

Agregada	80,62 % (3616/4485)
Código	80,45 % (3543/4404)
FSM	90,12 % (73/81)
Funcional	n/a

Ressalvas

- Teste dirigido em SystemVerilog, não a regressão UVM.
- Sem modelo de cobertura funcional (detalhado em E06).

▶ **Evidências E05 no GitHub**

E06 — Cobertura funcional do RSA_APB

Entregável

Relatório de cobertura funcional da interface RSA_APB de 32 bits

Requisito de aceitação

Métricas apresentadas e correlacionadas ao plano de verificação

---

Pergunta da revisão: *Qual é a taxa de cobertura funcional alcançada pelo ambiente de verificação UVM? Correlacione os itens do plano de verificação com as métricas de cobertura coletadas.*

E07 — Gate-level netlist TSMC 28 nm

Entregável

Gate-level netlist mapeado para a tecnologia TSMC de 28 nm

Requisito de aceitação

Netlist gerado corretamente pelo Cadence Genus

Pergunta da revisão: *Mostre-me o netlist do RSA_APB sintetizado utilizando a biblioteca de células da TSMC de 28 nm.*

E08 — Análise estática de temporização

Cadence Genus 22.18 · TSMC 28 nm · tt1v25c

Entregável

Relatório de análise estática de temporização (STA)

Requisito de aceitação — ATENDIDO

Ausência de violações de setup nos caminhos analisados (TNS = WNS = 0)

Pergunta da revisão: *O RSA_APB apresenta violações de timing? Mostre os relatórios e explique os resultados.*

SEM VIOLAÇÕES DE SETUP

WNS = +5 469,7 ps · TNS = 0 · 0 caminhos violados

Síntese	`2026-11-05_14-36-09`
Clock `pclk_i`	10 ns (100 MHz), incerteza 200 ps
I/O	`input_delay` = `output_delay` = 2 ns
Corner	tt1v25c · fio *segmented* (pré-layout)
Caminhos detalhados	100, todos **MET**
atraso externo / setup	33 / 67

▶ Evidências E08 no GitHub

E08 — Análise estática de temporização

Caminho crítico e interpretação

Path 1: `paddr_i[6]` → `pslverr_o`

Borda de captura	10 000 ps
− `output_delay`	2 000 ps
− incerteza	200 ps
Tempo requerido	7 800 ps
`input_delay`	2 000 ps
+ caminho de dados (19 células)	330 ps
Tempo de chegada	2 330 ps
Slack	**+5 470 ps**

Decodificação de endereço do APB (`NR4D0`, `ND2D1`, `AOI21...`) até a saída de erro.

- Os 33 piores caminhos são de **I/O**: o orçamento de 2 ns + 2 ns das restrições externas domina.
- Pior verificação de *setup* em registrador: `paddr_i` → `regbank_inst_exp_reg[18]/E`, slack **+7 338 ps**.
- Área: 2 808,16 μm², 2 562 células (706 seq. / 1 856 comb.).

Ressalvas

- Apenas *max delay* (setup); **hold não analisado**.
- STA de síntese: sem CTS, sem parasitas de layout, um único corner (TT).

▶ Evidências E08 no GitHub

E09 — Simulação pós-síntese com SDF

Gate-level TSMC 28 nm · Xcelium 24.03 · 100 MHz

Entregável

Relatório de simulação pós-síntese da interface RSA_APB, com anotação de atrasos SDF

Requisito de aceitação — ATENDIDO

Testes aprovados, sem violações de temporização reportadas durante a simulação

Pergunta da revisão: *Execute uma regressão dos testes do RSA_APB com o gate-level netlist e anotação SDF.*

TEST PASS — 398 / 398

0 *Timing violation* · 0 mensagens *E/*F · exit 0

Simulação	`2026-11-05_14-37-30`
Netlist / SDF	síntese `2026-11-05_14-36-09` (E08)
Biblioteca	`tcbn28hpcplusbwp30p140`
Corner	TT · 1,0 V · 25 °C · SDF `MAXIMUM`
Defines	`POST_SYNTHESIS`, `NTC`, `RECREM`
Tempo simulado	134 980 ns

▶ Evidências E09 no GitHub

E09 — Simulação pós-síntese com SDF

Anotação SDF, avisos e correlação com o RTL

Estatísticas de anotação

Categoria	Modelo	Anotados	%
Atrasos de caminho	14 871	14 871	**100,00**
`$setuphold`	7 314	7 312	99,97
`$recrem`	140	140	100,00
`$width`	10 372	0	0,00
Checks (total)	17 826	7 452	41,80

100 % do que o SDF fornece foi anotado. O SDF não contém entradas `WIDTH`; os 2 `$setuphold` restantes são do FF de sincronização do reset com `D = 1'b1`.

RTL (E05) × gate-level (E09)

As 398 linhas dos dois scoreboards são idênticas: valores esperados, lidos, status e instantes de verificação.

Ressalvas

- Avisos `NTCNNC` (132) e `NTCWID` (196): checks de pulso desativados.
- Um teste dirigido, não a regressão UVM completa.
- Sem layout nem parasitas extraídos; só o corner TT.

▶ **Evidências E09 no GitHub**

E10 — Scripts de automação

Entregável

Scripts de automação do fluxo de desenvolvimento

Requisito de aceitação

Execução reproduzível das etapas e geração dos respectivos artefatos

Pergunta

Quais funcionalidades ou cenários previstos ainda não foram exercitados pelos testes?